

Exercises week 3

Last update: 2026/09/06

The goal of these exercises is to give you practical experience on:

- Using the Java memory model to predict the value of program variables on an execution.
- Using the Java memory model to detect data races in concurrent programs.
- Using the Java memory model to ensure that synchronization primitives solve concurrency errors such as data races.

The code snippets for the exercises below are excerpts of the complete executable programs. They are complete code is located in the `code-exercises` folder for this week. We provide the complete versions, in case you would like to run the programs and observe their behavior. Furthermore, this shows that the concepts discuss in this lecture are applicable to executable Java code.

Exercise 3.1 Consider the program in file `CountingThreads.java`. The program contains the following thread class where `count` is a variable in shared memory.

```
public class CountingThread extends Thread {
    public void run() {
        int temp = count;
        count = temp + 1;
    }
}
```

The main thread of the program initializes the shared variable to 0, starts two threads, waits for their execution to finish and prints the resulting value.

```
count = 0; // Init shared variable

CountingThread t1 = new CountingThread();
CountingThread t2 = new CountingThread();

t1.start();
t2.start();

t1.join();
t2.join();

System.out.println("count="+count);
```

Mandatory

1. List the possible actions in the executions of the program. Classify the actions as: variable access, synchronization, or other. You may assign numbers to operations in the program to easily refer to them in this exercise and the ones below.
2. Define the happens-before order set containing action pairs obtained from the program order rule. Is this set the same for all possible program executions?
3. Define the happens-before order set containing action pairs obtained from the thread start rule and the thread termination rule. Is this set the same for all possible program executions?
4. Define the set of all possible synchronization orders for this program.
5. Show, using the Java memory model, that this program contains data races.

6. Use a lock to eliminate the data races in the program. Explicitly list the changes in program order and happens-before order compared to what you answered in the previous parts of the exercise. You do not need to explicitly write the new set of synchronization orders, but you must argue whether the possible executions in the new set of synchronization orders gives rise to more than one happens-before order set.
7. Show, using the Java memory model, that your updated program (3.1.6) does not contain data races.
8. Consider again the program without the lock, but defining `x` as `volatile`. Are all executions of this version of the program data race free? If you answer yes, use the Java memory model to show that the program does not contain data races. If you answer no, use the Java memory model to show the program contains data races.

Challenging

9. Here you will compare the possible values of the variable `count` for the lock-based version of the program (3.1.6) and the volatile based version (3.1.8). Use the Java memory model to show that, when the main thread executes `System.out.println("count="+count)`, these two programs may print different values of the variable `count`.
10. Formulate the mutual exclusion property for this program using the happens-before order.
11. Consider a modification of the data race free version of `CountingThread` where the increment of `count` is placed within an infinite loop such as `while(true)`. Note that this change leads to infinite executions. Assume that access to the critical section is protected by a lock. Show that this version of the program is data race free.

Exercise 3.2 In this exercise, the goal is to use the memory model to analyze a more realistic piece of Java code. Concretely, you will look into the class `StringSet` below.¹

```
public class StringSet {

    private final List<String> list = new ArrayList<String>();

    public synchronized int findOrAdd(String s) {
        int ret = list.indexOf(s);
        if (ret == -1) {
            list.add(s);
        }
        return ret;
    }

    public int find(String s) {
        return list.indexOf(s);
    }
}
```

Consider the concurrent program below that calls the two methods of the class concurrently.

```
StringSet s = new StringSet();

Thread t1 = new Thread(() -> {
    s.findOrAdd("PCPP");
});
Thread t2 = new Thread(() -> {
    s.find("PCPP");
});
t1.start();
t2.start();
```

¹The `StringSet` class is a simplification of the class `StringNumberer.java` (<https://github.com/soot-oss/soot/blob/develop/src/main/java/soot/util/StringNumberer.java#L36>) used in the Soot project (<https://github.com/soot-oss/soot>).

Mandatory

1. Show, using the Java memory model, that the program is *not* correctly synchronized.

NOTE I: In your reasoning, consider actions involving the method call `list.indexOf(...)` as a read access to `list`, and the method call `list.add(...)` as a write access to `list`.

NOTE II: The method calls `s.findOrAdd("PCPP")` and `s.find("PCPP")` are equivalent to expanding the method definition within the body of the thread.

2. Add the modifier `synchronized` to the definition of the method `find(...)` in the class `StringSet`. Show, using the Java memory model, whether the program is correctly synchronized. The notes for 3.2.1 also apply in this exercise.

Challenging

3. Does the data race that you found in 3.2.1 occur in all possible executions of the program? Use the Java memory model to justify your answer.

Exercise 3.3 Consider the following program:

```
int x = 0;

Thread t1 = new Thread(() -> {
    while (x==0)
        /*Do nothing*/
    System.out.println("t1: x="+x);
});
t1.start();
x=42;
System.out.println("Main: x="+x);
```

Mandatory

1. Last week we saw that (a similar version of) this program can loop forever, i.e., there exist executions where the operation `while (x==0)` is executed forever by thread `t1`. What pair of actions must be ordered by happens-before to prevent these infinite executions?
2. Show, using the Java memory model, that the pair of actions you identified in 3.3.1 are not ordered by happens-before in all executions of the program.

Challenging

3. Modify the program so that infinite executions are not possible. Show, using the Java memory model, that the actions you identified in 3.2.2 are ordered by happens-before in the modified program.